PROJECT CASE STUDY · APRIL 2026

Artistic Sisters

A full-stack art marketplace and commission platform built on .NET 10 microservices and Angular 21, with event-driven logistics and real payment processing.

ARCHITECTURE	BACKEND	FRONTEND	STATUS
Microservices + CQRS	.NET 10	Angular 21	Final Submission

01 — OVERVIEW

What Is Artistic Sisters?

Artistic Sisters is an online marketplace built around the handmade art business. It lets customers browse a gallery of artworks, place orders, and also request custom commissions directly from artists. On the other side, artists manage their portfolio and review commission requests. Delivery agents handle logistics — they see their assigned shipments and update the delivery status in real time.

The platform was built to go beyond a basic CRUD app. It uses a proper microservices architecture where each part of the system — orders, payments, logistics, notifications — runs as a completely independent service. This makes the system more resilient: if the notification service goes down, order placement still works.

6	12	7	3
Microservices	Integration Events	Order States	User Roles

14
Unit Tests

02 — PROBLEM

What Problem Does It Solve?

The fine arts market is a difficult space to digitise. Unlike regular e-commerce where you pick a product and buy it, art can be one-of-a-kind, hand-crafted to order, or negotiated between buyer and artist. Most existing platforms are built for mass-market goods and don't handle any of this well.

Specific Gaps Addressed

- **Two different order types in one flow:** Ready-made artwork that ships immediately needs a completely different workflow than a custom commission that requires artist approval and a production window. Both had to coexist cleanly in the same system.
- **Keeping data consistent across services:** When an order is placed, the payment service needs to know, the logistics service needs to prepare, and the customer needs a confirmation email — all without these services calling each other directly.
- **Role separation:** Artists should only see their own portfolio. Delivery agents should only see their assigned shipments. Customers should only see their own orders. This needs to be enforced server-side, not just in the UI.
- **Reliable notifications:** An email that never arrives for a confirmed order is a bad user experience. The system needed to guarantee that events like "order confirmed" or "agent assigned" always trigger their notifications, even if the broker hiccups temporarily.

03 — ARCHITECTURE

System Architecture

All client traffic goes through a single entry point — the Ocelot API Gateway on port 5000 — which routes requests to the right microservice. Every service has its own SQL Server database; no two services share a database. Cross-service communication happens exclusively through RabbitMQ events.

Port Map

COMPONENT	PORT	ROLE
Angular SPA	:4200	Frontend client
Ocelot Gateway	:5000	Single API entry point
Identity Service	:5001	Auth, JWT, users
Artwork Service	:5002	Gallery catalog
Order Service	:5003	Orders, commissions
Logistics Service	:5004	Shipments, agents
Payment Service	:5005	Razorpay integration
Notification Service	:5006	Email dispatch
RabbitMQ	:5672 / :15672	Message broker + admin UI
Redis	:6379	Order reservation cache
SQL Server	:1433	6 isolated databases

Microservice Responsibilities

:5001	**Identity Service** Handles all registration and login flows for customers and delivery agents. Issues 24-hour JWT tokens signed with HMAC-SHA256. Passwords are always stored as BCrypt hashes — never in plain text.
:5002	**Artwork Service** Manages the artwork catalog. Artists upload pieces with details like medium, price, dimensions, and available quantity. Customers can subscribe to "Notify Me" alerts for coming-soon items.
:5003	**Order Service** The most complex service. Manages the full order lifecycle through 7 states. Handles both standard ready-made orders and custom commission requests. Uses Redis for reservation state and an Outbox pattern for reliable event delivery.
:5004	**Logistics Service** Assigns delivery agents and vehicles to shipments when an order is ready for dispatch. Tracks delivery milestones and runs a Dispatch Saga for agent assignment with SLA monitoring.
:5005	**Payment Service** Integrates with the Razorpay SDK to create payment orders and verify payment signatures. Also supports Cash on Delivery. Listens to order events to initiate and confirm payment flows.
:5006	**Notification Service** A standalone email dispatcher. It doesn't expose any business endpoints — it just listens to RabbitMQ events and sends the right email to the right person. Customers, artists, and agents are all notified at relevant milestones.

Clean Architecture & CQRS

Every backend service follows Clean Architecture. The domain layer — entities, enums, business rules — has zero dependencies on any framework. Infrastructure concerns like EF Core and MassTransit live in their own layer and depend inward on domain abstractions.

All operations go through MediatR. A controller never directly calls a repository. It dispatches a command or query, and MediatR routes it to the right handler. This keeps controllers thin and testable, and makes business logic easy to find.

How a Request Flows Through the System

1. **Gateway** — Ocelot matches the route and forwards to the correct service.
2. **JWT Middleware** — Token is validated, role claims are extracted into the request context.
3. **Controller** — Binds the request body, checks `[Authorize(Roles="...")]`, dispatches via `IMediator.Send()`.
4. **FluentValidation** — Validates the command before the handler runs. Bad input fails here with a structured 400 response.
5. **MediatR Handler** — Runs business logic, mutates domain entities, prepares events.
6. **EF Core + Outbox** — Domain changes and the outbox message are written in one SQL transaction.
7. **MassTransit Relay** — Picks up the outbox message and publishes it to RabbitMQ for consumers.

Example: Registration Handler

IdentityService / Application / Commands / Register / RegisterHandler.cs

```
public async Task<RegisterResult> Handle(RegisterCommand request,
CancellationToken ct)
{
    // Reject duplicate emails before doing anything
    var exists = await _db.Customers.AnyAsync(c => c.Email ==
request.Email, ct);
    if (exists)
        return new RegisterResult { Success = false, Message = "Email
already registered" };

    // Password is always hashed — plain text never touches the database
    var customer = new Customer {
        Id           = Guid.NewGuid(),
        PasswordHash = BCrypt.Net.BCrypt.HashPassword(request.Password),
        IsActive     = true,
        RegisteredAt = DateTime.UtcNow
        // ... other fields
    };
    _db.Customers.Add(customer);
    await _db.SaveChangesAsync(ct);

    // Publish event so NotificationService sends the welcome email
    await _publisher.Publish(new CustomerRegisteredEvent {
        CustomerId = customer.Id, Name = customer.Name, Email =
customer.Email
    });

    return new RegisterResult { Success = true, Token =
GenerateToken(customer) };
}
```

Integration Events & Messaging

Services never call each other over HTTP. Everything is asynchronous through RabbitMQ via MassTransit. When something meaningful happens — an order is placed, an agent is assigned, a delivery is completed — the responsible service publishes an event. Any service that cares about that event subscribes to it independently.

Event Inventory

EVENT	PUBLISHED BY	CONSUMED BY	WHAT HAPPENS
`CustomerRegisteredEvent`	Identity	Notification	Welcome email to customer
`DeliveryAgentRegisteredEvent`	Identity	Notification	Onboarding email to agent
`CustomCommissionEvent`	Order	Notification	Commission request email to artist
`OrderPlacedEvent`	Order	Payment, Notification	Razorpay flow initiated + confirmation email
`ReadyForDispatchEvent`	Order	Logistics, Payment	Shipment created, agent assignment starts
`AgentAssignedEvent`	Logistics	Notification	Agent name & ETA emailed to customer
`SLAAtRiskEvent`	Logistics	Notification	SLA breach warning emailed to admin

EVENT	PUBLISHED BY	CONSUMED BY	WHAT HAPPENS
`OrderStatusChangedEvent`	Order	Notification	Status update email to customer, agent & admin
`DeliveryCompletedEvent`	Logistics	Notification	Delivery confirmation, agent released
`DispatchConfirmedEvent`	Logistics	Notification, Payment	Dispatch confirmation email
`AssignAgentFailedEvent`	Logistics	Payment	Compensation / retry logic triggered
`CustomerApprovedEvent`	Identity	Notification	Account activation email

The Outbox Pattern — Why It Matters

There's a well-known problem in distributed systems: what if you successfully save to the database but then fail to send the RabbitMQ message? The order exists but no one gets notified. The Outbox pattern solves this.

In the Order Service, when a handler runs, it writes the domain change *and* the serialised event into an `OutboxMessages` table — all in the same SQL transaction. A background relay process polls this table and publishes the messages to RabbitMQ, marking them as processed once done. If the relay crashes mid-flight, it simply retries — the message is still in the outbox.

> **Result:** At-least-once delivery is guaranteed. No order confirmation, payment, or logistics event can go missing due to a transient broker failure.

06 — ORDER LIFECYCLE

Order State Machine

Every order follows a strict sequence of states defined in the `OrderStatus` enum. The `UpdateOrderStatusHandler` enforces valid transitions — you can't jump from Pending straight to Delivered. Each transition publishes one or more events.

Pending → Confirmed → In Production → Ready for Delivery → Out for Delivery → Delivered / Cancelled

Key Transition Logic

TRANSITION	EVENTS PUBLISHED	EFFECT
→ Confirmed	`OrderStatusChangedEvent`	Customer gets "Order confirmed" email
→ In Production	`OrderStatusChangedEvent`	Customer gets "Work has started" email
→ Ready for Delivery	`ReadyForDispatchEvent` + `OrderStatusChangedEvent`	Logistics creates shipment and assigns an agent
→ Out for Delivery	`OrderStatusChangedEvent`	Customer gets "Your package is on the way" email
→ Delivered	`OrderStatusChangedEvent`	Customer gets delivery confirmation, agent is freed
→ Cancelled	`OrderStatusChangedEvent`	Customer gets cancellation email

Idempotency guard: The handler checks whether the order is *already* in `ReadyForDelivery` before publishing `ReadyForDispatchEvent`. If someone re-submits the same status update, no second dispatch event fires — preventing duplicate agent assignments.

07 — CUSTOM COMMISSIONS

Commission Workflow

Custom commissions are the differentiator of this platform. Unlike a standard purchase, a commission requires a back-and-forth between the customer and artist before any payment happens.

1. **Customer submits request** via `POST /api/orders/commission` — providing medium, size, instructions, and optionally a reference photo URL. The order is created with status `Pending`.
2. **Artist is notified** via the `CustomCommissionEvent` that triggers an email from the Notification Service with all the commission details.
3. **Artist reviews on their dashboard** and either accepts (sets a quoted price, moves to `Confirmed`) or declines (`Cancelled`). In both cases, an `OrderStatusChangedEvent` emails the customer.
4. **Customer accepts and pays** — once confirmed with a price, the customer proceeds through the normal checkout: Razorpay online payment or Cash on Delivery.
5. **Standard fulfilment** — the order then follows the same In Production → Ready → Dispatch → Delivered pipeline as any other order.

Angular 21 Frontend

The frontend is a standalone Angular 21 SPA. It uses reactive forms, RxJS streams, and a JWT interceptor that automatically attaches `Authorization: Bearer <token>` to every API request. The token, role, and userId are stored in localStorage under the keys `as_token`, `as_role`, and `as_userId`.

Route Guard Matrix

ROUTE	GUARD	ALLOWED
`/about`, `/portfolio`	None	Public
`/auth/login`, `/auth/register`	None	Public
`/order`, `/commission`, `/track/:id`	`authGuard`	Any logged-in user
`/customer/dashboard`	`authGuard`	Customer
`/artist`	`artistGuard`	Artist
`/delivery/dashboard`	`deliveryGuard`	Delivery Agent
`/admin/dashboard`	`adminGuard`	Artist (admin)

Guards check the role stored in localStorage. If the role doesn't match, the user is redirected to `/auth/login`. Server-side, the `[Authorize(Roles="...")]` attribute provides a second enforcement layer — so even if someone bypasses the frontend guard, they can't access protected API endpoints.

09 — SECURITY

Security Model

LAYER	MECHANISM	PURPOSE
Password storage	BCrypt (cost factor 10)	Protects against rainbow table attacks
Authentication	HMAC-SHA256 JWT — 24h TTL	Stateless, verifiable identity on every request
API authorisation	`[Authorize(Roles="...")]` — server-side	Prevents cross-role API access
Frontend navigation	4 Angular route guards	Prevents unauthorised page access
HTTP layer	JWT interceptor on all requests	Auto-attaches bearer token from localStorage
Payment	Razorpay signature verification	Prevents payment tampering

Unit Tests

The project includes 14 unit tests across two test projects — `IdentityService.Tests` and `OrderService.Tests`. All tests use EF Core InMemory databases so they need no real SQL Server, Moq for mocking MassTransit's publish endpoint, and xUnit as the runner. Every test is fully isolated and deterministic.

IdentityService Tests (5 tests)

- **`Handle_NewEmail_ReturnsSuccess`**
  Registering with a new email returns `Success = true`, a non-null JWT, and a valid CustomerId.
- **`Handle_DuplicateEmail_ReturnsFailure`**
  Re-registering with an existing email returns `Success = false` and the exact message "Email already registered".
- **`Handle_NewEmail_PersistsCustomerInDatabase`**
  After registration, the customer record exists in the DB with the correct name and `IsActive = true`.
- **`Handle_NewEmail_PublishesCustomerRegisteredEvent`**
  MassTransit's `Publish<CustomerRegisteredEvent>` is called exactly once after successful registration.
- **`Handle_NewEmail_StoresPasswordAsHash`**
  The stored `PasswordHash` is never equal to the plain-text password, and `BCrypt.Verify()` confirms the hash is valid.

OrderService Tests — PlaceOrder Handler (7 tests)

- **`Handle_ValidOrder_ReturnsSuccess`**
  A valid command returns success with a non-null OrderId.
- **`Handle_ValidOrder_PersistsOrderAndItems`**
  The Order and its OrderItems are written to the DB with correct data, including artwork title matching.
- **`Handle_ValidOrder_CalculatesTotalAmountCorrectly`**
  Two-item order (3×₹200 + 1×₹100) produces `TotalAmount = 700`. Server-side price calculation is never trusted from the client.

- **`Handle_EmptyItems_ThrowsDomainException`**
  An order with no items throws a `DomainException` before any DB write happens — domain invariant enforced correctly.

- **`Handle_ValidOrder_PublishesOrderPlacedEvent`**
  `OrderPlacedEvent` is published exactly once to trigger the payment and notification pipeline.

- **`Handle_ValidOrder_SetsStatusToConfirmed`**
  New card/COD orders start with `Confirmed` status, not `Pending` — as per business rules.

- **`Handle_ValidOrder_WritesOutboxMessage`**
  A single `OutboxMessage` of type `OrderPlacedEvent` is written in the same transaction as the Order record.

OrderService Tests — UpdateOrderStatus Handler (extra 2 tests)

- **`Handle_StatusToReadyForDelivery_PublishesDispatchEvent`**
  Transition from InProduction → ReadyForDelivery fires both `ReadyForDispatchEvent` and `OrderStatusChangedEvent` exactly once each.

- **`Handle_AlreadyReadyForDelivery_DoesNotPublishDispatchEvent`**
  Idempotency: re-submitting the same ReadyForDelivery status does NOT fire a second dispatch event. Prevents duplicate agent assignment.

Libraries & Packages

PACKAGE	VERSION	USED FOR
.NET Runtime	10.0	All backend microservices
MediatR	12–14.x	CQRS command/query dispatching
MassTransit + RabbitMQ	8.0.11	Event publishing and consuming
Entity Framework Core	8.0–10.0	Code-first ORM, migrations
Ocelot	24.1.0	API Gateway routing
FluentValidation	11.7.0	Request validation (OrderService)
BCrypt.Net-Next	4.0.2	Password hashing (IdentityService)
JwtBearer	8.0.16	JWT authentication middleware
StackExchange.Redis	2.12.8	Reservation cache (OrderService)
Razorpay SDK	3.3.2	Payment processing (PaymentService)
Angular	^21.2.0	Frontend SPA framework
xUnit + Moq	Latest	Unit testing

Key Takeaways

Artistic Sisters is not a prototype or a CRUD demo. It is a working, end-to-end platform with real payment processing, a proper microservices architecture, a tested business layer, and a functioning frontend across three distinct user roles.

The main engineering decisions — CQRS with MediatR, event-driven communication over RabbitMQ, the Outbox pattern for reliability, database-per-service isolation, and BCrypt + JWT for security — were all made deliberately to build something that would hold up in a production environment, not just in development.

- Services are independently deployable — one failing doesn't take down the rest.
- Events are never silently lost — the Outbox pattern guarantees delivery.
- Business rules are enforced server-side — the frontend guards are a UX layer, not the security layer.
- Every critical handler is covered by a focused, isolated unit test.
- Custom commissions and standard orders share the same core pipeline, keeping the codebase unified.